

Assignment 2

Artificial Neural Networks for Regression

© Industrial Analytics Group
 Dept. Civil and Industrial Engineering, Uppsala University
 November 4, 2025

Contents

1	Goals	3
2	Introduction	4
2.1	Introduction - FANN models with multiple inputs	4
2.2	Specify your FANN architecture:	4
2.3	Parameter initialization	4
2.3.1	Example 1: A linear regression model viewed as a single neuron with a linear activation function	4
2.3.2	Example 2: A FANN with 2 inputs, 2 hidden layers and 1 output	5
2.3.3	Example 3: A FANN with 3 inputs, 2 hidden layers and one output	6
2.4	Task A - 1D regression using a simple FANN	6
2.4.1	Main differences from standard parameter fitting	6
2.4.2	Storing a dataset as a PyTorch “TensorDataset”	6
2.4.3	The PyTorch “DataLoader”	7
2.4.4	Workflow template	7
2.4.5	Exercises	8
3	Theory I: Some general theory and tips	10
3.1	Single artificial neurons	10
3.2	Multilayer FANNs - also known as Multilayer Perceptrons	11
3.3	The Root Mean Squared Error	12
3.4	Stochastic Gradient Descent, Mini-Batches and Epochs	12
4	Theory II: Building and training ANNs in PyTorch	14
4.1	The PyTorch object Dataset	16
4.2	Already prepared/preloaded PyTorch Datasets	16
4.3	Splitting your data into ‘train-’, ‘validation-’ and ‘test-’ -sets	17
4.4	Define and construct fully connected FANN models in PyTorch	17
4.4.1	Specify your FANN architecture:	18
4.4.2	Parameter initialization	18
4.4.3	Example 1: A FANN with 2 inputs, 2 hidden layers and 1 output	18
4.4.4	Example 2: A FANN with 3 inputs, 2 hidden layers and one output	18
4.5	How to feed data into your ANN models to make predictions	19
4.6	Simplified data management using “DataLoader”	19
4.7	Simplified updating in PyTorch using “optimizer”	19
4.8	Model performance evaluation - during and after training	20
4.9	Putting it all together: The flow of ANN training	21
5	Task B: Non-linear univariate regression using FANNs	22
6	Task C: Approximate a bell-shaped bivariate function	23
7	Task D: Model selection using a validation set	26

8 Task E: Early Stopping Using a Validation Set	27
9 DEADLINE AND EXAMINATION	28
10 Extra Study Material, Tips, Etc	29
10.1 PyTorch Help, Troubleshooting and Guidance	29
10.2 Gradient Decent and Stochastic Gradient Decent	29
10.3 Backpropagation to achieve auto-differentiation	29

1 Goals

The overarching aims here are to learn about:

- what is a feedforward artificial neural network (FANN) and the surrounding theory and terminology
- how a single artificial neuron operates by first computing a weighted sum, then subtracting a threshold value (or adding a bias), and finally applying a linear or non-linear activation function to produce its output.
- the power of Stochastic Gradient Descent using *mini-batches*, compared to conventional *epoch* based Gradient Descent
- how sets of non-overlapping training, test and validation examples must be created and organized in order to use them in the PyTorch framework and similar frameworks.
- how the PyTorch framework works when it comes to implement and train FANNs for regression problems.
- the importance of splitting the set of examples in the original dataset available into at least two parts, where the second part is only used to obtain an **unbiased (neutral) performance estimation** of the final ANN built. This final ANN must only be selected and trained using the first part of the original dataset. Typically this final ANN has been selected and trained after first having tried multiple network structures, learning rates etc using the exclusively the examples in first part of the dataset.
- **early stopping** as a mean reduce/avoid overfitting. By stopping the fitting when the performance on the validation set starts to decrease, it is possible to reduce/avoid the risk of fitting the model parameters too much to the particular set of training examples available.

2 Introduction

2.1 Introduction - FANN models with multiple inputs

In this subsection you will learn how to define *custom architectures* of ANNs in PyTorch. By architecture, one simply means the number of inputs, the number of hidden layers, the number of output neurons, the number of neurons/nodes in each layer, the type of activation functions, etc. In other words, the architecture defines every module and operation involved in processing an input $\mathbf{x}_n$ into an output $\hat{y}_n$.

Remark

There are two ways to specify and build your ANN architecture, either by creating a PyTorch specific class for your network, or by using the more simpler but less flexible `nn.Sequential(...)` method. Here you will only need to be able to construct your ANNs using the second (simpler) approach.

2.2 Specify your FANN architecture:

The general procedure of defining your FANN architecture using the model constructor `nn.Sequential(...)` is by specifying all the features of the network that your data will flow through. These features are mainly the number of layers, the number of nodes (neurons) in each layer, and the activation functions of these nodes.

2.3 Parameter initialization

Every time you initialize a new FANN model in PyTorch, that model will have its parameters to be fitted (weights and biases) set to fresh randomly selected small values.

2.3.1 Example 1: A linear regression model viewed as a single neuron with a linear activation function

In this example we want to create an ANN that is nothing more than a linear parametric model that takes in 10-dimensional input vectors $\mathbf{x}_n$ and returns a scalar value that is a weighted sum of the elements in input vector $\mathbf{x}_n$ defined by parameter vector $\mathbf{w}$, also with an bias w_0 added. In other words, we want to build an ANN corresponding to the following mathematical model:

$$y(\mathbf{x}; \mathbf{w}) = w_0 + w_1x_1 + w_2x_2 + \dots + w_{10}x_{10} = w_0 + \sum_{i=1}^{10} w_ix_i \quad (1)$$

Visually, the network diagram in Fig. 1 corresponds to the above model where the input layer of 10 nodes represents the *input layer*, meaning the input $\mathbf{x}$ and the connections represent the elements w_i of $\mathbf{w}$. The single node to the right is the output node representing the output value of the network $\hat{y}$. Notably, the addition of the term w_0 is not visible in this figure.

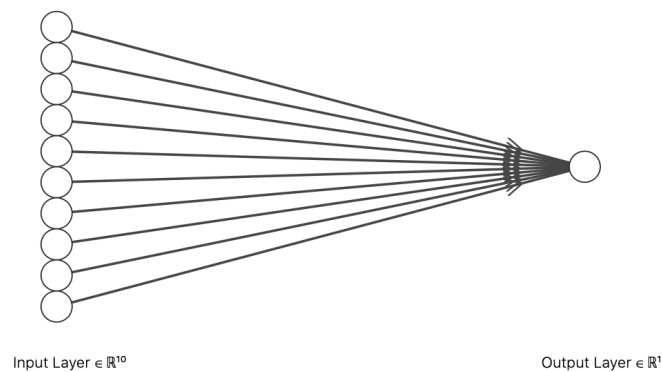


Figure 1: A single artificial neuron with 10 inputs having no non-linear activation function. Such a simple ANN model is equivalent to a linear regression model. Note: The additive bias term is not visible in this network diagram.

The code needed to create this network in PyTorch is presented here:

```
1 # Defining a single neuron having 10 inputs
2 model = nn.Sequential(
3     nn.Linear(in_features=10, out_features=1)
4 )
```

2.3.2 Example 2: A FANN with 2 inputs, 2 hidden layers and 1 output

For an illustration, see Fig.2. The activation function *ReLU* should be used for the hidden layer neurons and the activation function *tanh* should be used at the output so that network output values are restricted to the interval $[-1, 1]$.

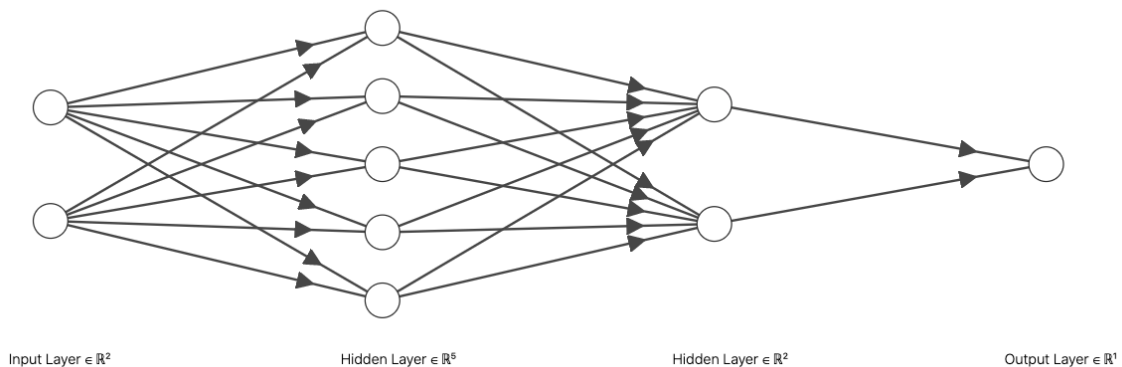


Figure 2: A FANN with 2 inputs, 2 hidden layers and 1 output.

The code needed to create this network in PyTorch is presented here:

```
1 # Simple way to define a Fully connected feed forward ANN with two inputs, 5
2 # neurons in the first hidden layer, 2 neurons in the second hidden layer
3 # and a single output neuron. NOTE: Different types of activation functions
4 # in each layer
5 model = nn.Sequential(
6     nn.Linear(in_features=2, out_features=5),
7     nn.ReLU(),
8     nn.Linear(in_features=5, out_features=2),
9     nn.ReLU(),
10    nn.Linear(in_features=2, out_features=1)
11    nn.Tanh()
12 )
```

2.3.3 Example 3: A FANN with 3 inputs, 2 hidden layers and one output

For an illustration, see Fig.3. Here the ReLU activation function should be used for all hidden neurons and no activation function should be employed for the output neuron.

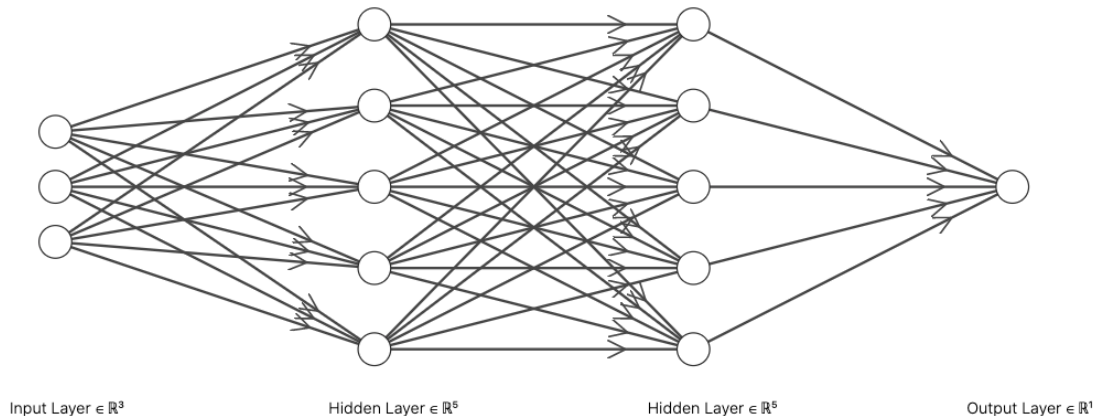


Figure 3: A FANN with 3 inputs, 2 equally large hidden layers and one output.

The code needed to create this network in PyTorch is presented here:

```
1 # Simple way to define a Fully connected feed forward ANN
2 model = nn.Sequential(
3     nn.Linear(in_features=3, out_features=5),
4     nn.ReLU(),
5     nn.Linear(in_features=5, out_features=5),
6     nn.ReLU(),
7     nn.Linear(in_features=5, out_features=1)
8 )
```

2.4 Task A - 1D regression using a simple FANN

The task here is to train one of the simplest possible type of FANN using a set of noisy data-points where a linear predictor is reasonable to use.

2.4.1 Main differences from standard parameter fitting

As shown here, the overall methodology for FANN training considered here is still more or less the same as for parameter fitting in simpler cases, but there are some important differences:

- We are now in general using much more flexible/powerful types of model families with many more parameters than when working with classical parametric models (that typically have less than 10 unknown parameters). However, here your job will be to define a super simple FANN that only consists of a single input for x_n and a single output neuron to predict the desired value y_n .
- We are replacing conventional Gradient Descent by “mini-batch” based **Stochastic Gradient Decent** when fitting the parameters of our model family to the data. This means that each iterative step is based on a random subset (mini-batch) of all training examples, instead of using all of them. Stochastic Gradient Decent comes with some very interesting advantages to be discussed later.
- Instead of calling it *parameter fitting* or *parameter estimation*, in the context of ANNs the corresponding procedure is often called **network training**.

2.4.2 Storing a dataset as a PyTorch “TensorDataset”

Before going to the general workflow, here follows a code snippet showing how to transform any generated/collected dataset into the PyTorch specific datatype **TensorDataSet**. This is needed on

order to use the PyTorch function `DataLoader`, which will simplify the network training by taking care of generating the mini-batches (to be discussed below). Just run the code and make sure you understand that it first generates only training examples using NumPy, one example per row, and then stores them in the form of the PyTorch specific `TensorDataset` format.

```
1 import numpy as np
2 import torch
3 from torch.utils.data import TensorDataset
4 from torch.utils.data import DataLoader
5
6 inputs = np.random.rand(3, 2) # Input data
7 targets = np.random.rand(3, 1) # Correct answer data
8
9 inputs_tch = torch.FloatTensor(inputs) # Change to an array that can be handled by pytorch
10 targets_tch = torch.FloatTensor(targets) # Same as above
11
12 print(inputs_tch)
13 print(targets_tch)
14
15 torch_dataset = TensorDataset(inputs_tch, targets_tch) # set the data
16 print("Content of torch_dataset=", vars(torch_dataset)) # vars prints the contents of the object
17
18 tuple_of_tensors=torch_dataset.tensors
19 print("tuple_of_tensors=", tuple_of_tensors)
20 list_of_tensors=list(tuple_of_tensors)
21 print("list_of_tensors=", list_of_tensors)
```

2.4.3 The PyTorch “DataLoader”

The `DataLoader` provided by PyTorch is used to simplify the coding related to training your network models. Here follows a small code snippet intended to show how the `DataLoader` generates mini-batches of size 2 from the dataset `torch_dataset` created above. Run this snippet after first running the previous code. Make sure you understand the idea that a mini-batch is just a subset of the original dataset. Also make sure you understand the fact that the `DataLoader` helps you generate all possible non-overlapping subsets (of size 2) from the original data set.

```
1 train_loader = DataLoader(torch_dataset, batch_size=2, shuffle=True) # Data shuffle with shuffle=
  True
2
3 for x, t in train_loader:
4     print(x)
5     print(t)
```

Notably, since we only have 3 examples (rows) in this dataset, although the parameter `batch_size` is set to the value 2, we actually get one mini-batch of size 2 and one of size 1.

You need also to note and understand the effect of setting the parameter `shuffle` True as used here. This means that the subsets created are drawn randomly (without replacement) from the original dataset. If you change from True to False, you will be able to see that the mini-batch subsets are created systematically (not randomly) from the original dataset. Regardless, you must understand that in both cases the generated mini-batch subsets are non-overlapping and exhaustive: This means that all examples in the original dataset are used to create these subsets, and that each example will occur only in one of the subsets created.

2.4.4 Workflow template

Here follows an example of a code snippet with a simplified FANN training workflow.

```
1 # CODE TEMPLATE FOR SIMPLE TRAINING OF A FANN IN PYTORCH
2 import torch
3 from torch import nn
4 from torch.utils.data import DataLoader, TensorDataset,
5 from torchvision import datasets
6 from torchvision.transforms import ToTensor,
7
8
9 # Make a DataLoader object to handle mini-batches for data.
10 # For example
11 # my_dataloader = DataLoader(dataset = train_dataset, batch_size = 32, shuffle = True )
12
13 # Define and construct your neural network architecture.
14 # For example:
15 # model = nn.Sequential(nn.Linear(in_features=1, out_features=5),
```

```

16 #                 nn.ReLU(),
17 #                 nn.Linear(in_features=5, out_features=1),
18 #                 nn.ReLU()
19 #             )
20
21 # Select a Cost function
22 # Select your PyTorch cost/loss function from torch.nn
23 # For example:
24 # cost_function = nn.MSELoss()
25
26 # Select Optimizer (optimization method) and give model parameters and learning rate
27 # For example
28 # optim = torch.optim.SGD(model.parameters(), lr=0.0001 )
29
30 list_of_minibatch_costs = [] #A list for storing values of the cost function
31 for epoch_i in range(nr_epochs): # Iterate through whole dataset nr_epochs times
32
33     for X_batch, y_batch in my_dataloader: # Iterate through a batch
34         y_preds = model(X_batch) # Make Prediction on mini-batch
35
36         cost = cost_function(y_preds, y_batch) # Compute cost for mini-batch
37
38         optim.zero_grad() # Set the grads of all model params to zero.
39
40         cost.backward() # compute and populate gradients of model params
41
42         optim.step() # instruct optimizer to take one update step
43
44     list_of_minibatch_costs.append(cost.item())
    
```

Remark. The command `cost.item()` converts the tensor value `cost` into a plain python number.

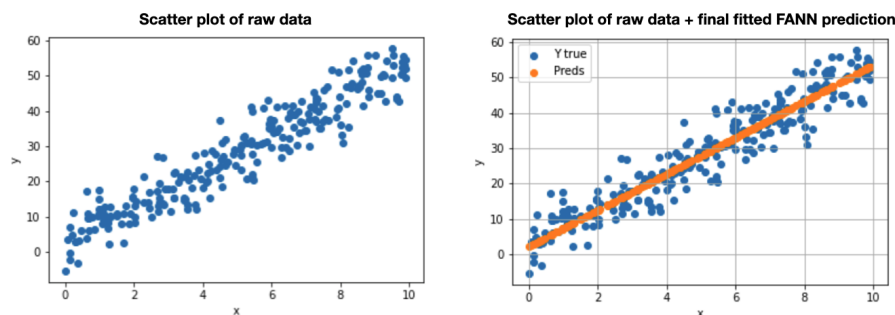


Figure 4: Left: First you should create our own synthetic dataset where you will know exactly what is the true functional relationship between the values x_n and y_n generated. You should also embed the generated response values y_n in zero mean additive noise. Right: Same data with predictions illustrated in orange.

2.4.5 Exercises

In the following exercises, you will fit your own *synthetic* dataset coming from an linear relationship between x and y , with all y -values embedded in zero mean Gaussian additive noise. The values of the two parameters (slope and intercept) defining the linear relationship, as well as the value of standard deviation of the additive noise, you should choose yourself. Thus you should begin by generating something similar to the left subfigure in Fig. 4 and then train your simple FANN using your generated data such that it can perform predictions like the orange curve/line in the right subfigure.

Problem 2.1 - Generate Your Own Data

You should generate and plot a dataset similar to the left subfigure in Fig. 4. For some help, here follows some template code. Pay close attention to lines 8 and 9 where we need to reshape our data so that each row is a single x_n . This is because when we feed our X into the model, the model spits out a prediction for each row, meaning a column vector of the same shape as y_{obs} .

```
1 a_true = 5.0 # choose your own value here
2 b_true = 3.0 # choose your own value here
3 sigma_epsilon = 5.0 #choose your own noise level here
4
5 X = np.random.uniform(0, 10, 250) # sample 250 values x_n in [0,10]
6 y_obs = a_true*X + b_true + sigma_epsilon*np.random.randn(len(X)) # linear relationship embedded
   in zero mean additive gaussian noise
7
8 #Convert to one observation per row and then convert into PyTorch tensors
9 X = torch.tensor(X).reshape(len(X),1).float()
10 y_obs = torch.tensor(y_obs).reshape(len(y_obs),1).float()
```

Problem 2.2 - Make a PyTorch dataset object “TensorDataset”

Based on your generated synthetic data, use `TensorDataset(X, y_obs)`. Then create a PyTorch `DataLoader` object that will automate your mini-batch selection.

This means you need to fill in the following code snippet:

```
1 train_dataset = TensorDataset(torch.tensor(train_x), torch.tensor(train_y))
2 my_dataloader = DataLoader(...)
```

You should choose the mini-batch size to be somewhere between 1% to 50% of the number of examples in your training data set. Moreover, do not forget to make sure that you are reshuffling (reordering) your mini-batches for every new epoch by setting the flag `shuffle True`.

Problem 2.3 - Create one-dimensional linear predictive model

Now create the following linear prediction model:

$$y(x, \mathbf{w}) = y(x, [w_0, w_1]) = w_0 + w_1 x \quad (2)$$

in your code. As shown in Fig. 5, this is the very simplest possible FANN model.



Figure 5: One-dimensional linear parametric model in the form of one of the simplest possible FANNs.

Problem 2.4 - Define cost function

The following definition will determine what type of cost function you should choose. Thus select the most naturally suitable cost function among the following alternatives.

```
1 # Alternative loss functions
2 cost_function = nn.MSELoss()
3 cost_function = nn.L1Loss()
```

Problem 2.5 - Choose optimizer and learning rate

As your optimizer object, choose the `torch.optim.SGD(...)` optimizer and feed in as arguments your model parameters, and choose some appropriate initial learning rate. Note: You might need to adjust the learning rate if your training does not seem to converge to a local minimum.

Problem 2.6 - Train your model and plot training cost curve

Train your model using mini-batch Stochastic Gradient Decent by implementing the main training loop. Keep track of the cost of each mini-batch associated with each parameter update.

```
1 training_minibatch_Js = []
2 nr_epochs = # Try
```

```
3 for epoch_i in range(nr_epochs):
4     for X_batch, y_batch in my_dataloader:
5         # perform gradient decent on mini-batch
6         # with your optimizer taking care of the actual updates
7         # of your model parameters.
8
9     # Plot your cost
10    plt.figure(figsize=[10,5])
11    plt.plot(training_minibatch Js)
12    plt.xlabel('update step i on mini-batch')
13    plt.ylabel('Cost')
14    plt.title('Cost during training on train-set (per mini-batch)')
15    plt.grid()
```

You should arrive at a training cost curve similar to the one in Fig. 6.

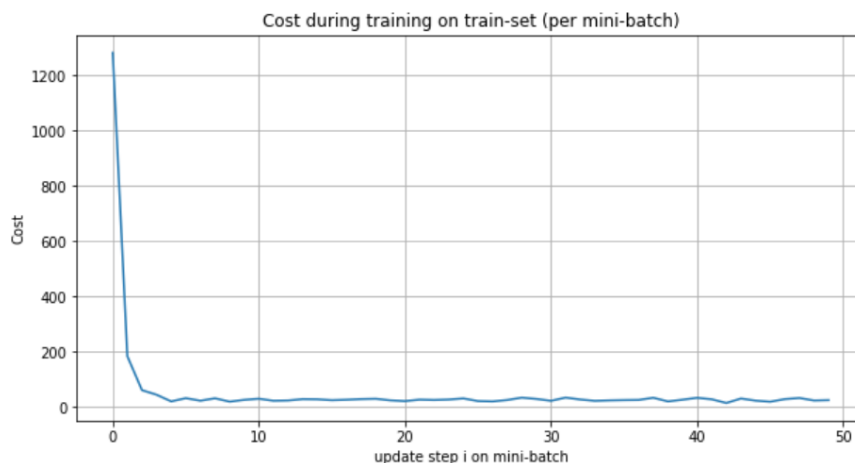


Figure 6: An example of how the cost based on the training set is decreasing as a function of the number of mini-batch updates performed during training.

Problem 2.7 - Make predictions

Based on the inputs x_n , plot both your data-points and your model predictions like in the right sub-figure in Fig. 4. Using your model outside of training you should not spend time and energy computing gradients anymore. Therefore you should turn the gradient calculations off by means of `with torch.no_grad():` as shown below.

```
1 with torch.no_grad():
2     y_preds = model(X)
```

Problem 2.8 - Print and look at your final model parameters

When having few parameters, it makes sense to print and look at their values after the training (parameter fitting) step. You can for example check if your training resulted in parameter values that agree with the true parameters you used to generate the training examples.

```
1 # Print all parameters in your neural network model
2 for p in model.named_parameters():
3     print(p)
```

3 Theory I: Some general theory and tips

3.1 Single artificial neurons

In the context of FANNs, the output y from a single artificial neuron with d inputs x_i is usually determined as $y = \varphi(v)$ where $v = w_0 + w_1x_1 + w_2x_2 + \dots + w_dx_d$ where $\varphi()$ is a nonlinear *activation function* defined for example as $\varphi(v) = 1/(1+e^{-v})$, $\varphi(v) = \max\{0, v\} = \text{ReLU}(v)$, or $\varphi(v) = \tanh(v)$.

Problem 3.1 - A single artificial neuron

- (a) Make a plot using Python that shows the four activation functions $\varphi(v) = 1/(1 + e^{-v})$, $\varphi(v) = \max\{0, v\}$, $\varphi(v) = \tanh(v)$, and $\varphi(v) = v$ on the interval $[-5, 5]$.
- (b) An artificial neuron with only one input value x has two adjustable parameters. Which are they?
- (c) How many adjustable parameters does a single neuron have when its input is a d -dimensional input vector $\mathbf{x} = [x_1, x_2, \dots, x_d]^T$?

3.2 Multilayer FANNs - also known as Multilayer Perceptrons

Fig. 7 illustrated a FANN with two inputs, 5 artificial neurons in the first hidden layer, 2 artificial neurons in the second layer, and one single output artificial neuron. This particular kind of networks are often called Multilayer Perceptrons. Notably, such networks do not have any direct connections to the output neuron(s) from the input or any of the hidden layers other than the final one. Having this kind of connections is indeed a possibility, but not considered any further here.

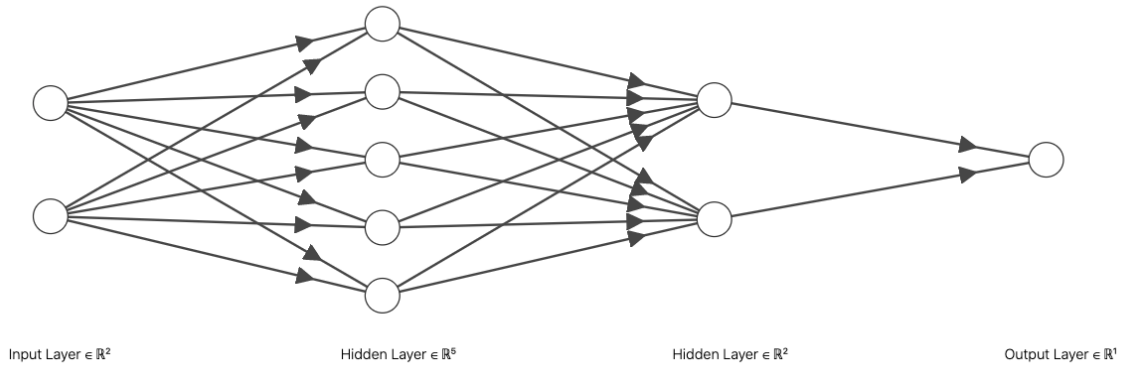


Figure 7: A FANN with two hidden layers of neurons. Notably, this network architecture does not have any direct connections to the output node(s) from the input or any hidden layer except for the last one.

Notation used: In order to describe a network with M hidden layers, a natural notation to use is I-H1-H2-...-HM-O. Here I denotes number of inputs, H_i the number of nodes in the i :th hidden layer, and O denotes the number of nodes in the output layer. Thus, for example 1-3-2 means a network with one input, one hidden layer having 3 nodes, and with two nodes in the output layer.

Problem 3.2: Number of tunable parameters in Multilayer FANN

Assuming that each neuron works as described in the previous subsection above (meaning a weighted sum plus a bias, followed by a non-linearity), how many tunable parameters does the network have?

Problem 3.3: Mathematical expression for a Multilayer FANN output

Let the weight from input i to neuron j in the first hidden layer be denoted by u_{ji} and let its bias term be denoted $u_{j,0}$. Similarly, let the weight from output i of the first hidden layer to neuron j in the second layer be denoted v_{ji} with bias term $v_{j,0}$. Finally, let the weights from output i of the second hidden layer to the output neuron be denoted $w_{\text{out},i}$ with bias term $w_{\text{out},0}$. Using this notation, assuming that all nodes are using the same activation function $\varphi()$, write down a mathematical expression (formula) for the network output $y(x_1, x_2)$ where x_1 and x_2 denote the network input.

Tip: You may start by writing

$$y(x_1, x_2) = \varphi(w_{\text{out},0} + w_{\text{out},1} \cdot z_1(x_1, x_2) + w_{\text{out},2} \cdot z_2(x_1, x_2))$$

where $z_1(x_1, x_2)$ and $z_2(x_1, x_2)$ are the two outputs from the second hidden layer. The final mathematical expression should be a function of (x_1, x_2) as well as of all weights and biases in the network.

3.3 The Root Mean Squared Error

Often when training ANNs for regression problems we use the mean squared error (MSE)

$$J_{MSE} = \frac{1}{N} \sum_{n=1}^N (y_n - \hat{y}_n)^2$$

as the objective function to be minimized. Here y_n is the desired (target) value and $\hat{y}_n$ is the corresponding network prediction for example n . However, a quantity which is more easily interpreted for a human is the square root of this performance measure. This is known as the Root Mean Squared Error (RMSE) and defined as

$$J_{RMSE} = \sqrt{\frac{1}{N} \sum_{n=1}^N (y_n - \hat{y}_n)^2}.$$

In practice this quantity is usually not used for training but only during final evaluation of the prediction model built. This is because it is much easier for a human to have a practical interpretation of the size of an RMSE value, compared to the corresponding mean squared error. This is because the RMSE is expressed in the same unit as the response y_n . Notably, since the square root function $\sqrt{x}$ is monotonic, the minima of J_{RMSE} and J_{MSE} will be the same.

Problem 3.4: Interpretability of MSE vs RMSE

Assume y_n has the unit kg. What will then be the unit of the MSE and the RMSE, respectively?

3.4 Stochastic Gradient Descent, Mini-Batches and Epochs

If the problem is to minimize the objective function

$$J(a, b) = (y_1 - a \cdot x_1 - b)^2 + (y_2 - a \cdot x_2 - b)^2 + (y_3 + a \cdot x_3 - b)^2$$

using Gradient Descent, we should move in the opposite direction of the gradient $\mathbf{g} = [g_1, g_2]^T$. Here the partial derivatives of the gradient are:

$$g_1 = \frac{\partial J}{\partial a} = -2(y_1 - a \cdot x_1 - b) \cdot x_1 - 2(y_2 - a \cdot x_2 - b) \cdot x_2 - 2(y_3 + a \cdot x_3 - b) \cdot x_3$$

$$g_2 = \frac{\partial J}{\partial b} = -2(y_1 - a \cdot x_1 - b) - 2(y_2 - a \cdot x_2 - b) - 2(y_3 + a \cdot x_3 - b)$$

Stochastic approximation: A *stochastic approximation* of the gradient is obtained if we use only one or two of these terms to approximate the partial derivatives, for example:

$$g_1 \approx \hat{g}_1 = -2(y_2 - a \cdot x_2 - b) \cdot x_2$$

$$g_2 \approx \hat{g}_2 = -2(y_2 - a \cdot x_2 - b)$$

Advantages: Moving in the opposite direction of $\hat{\mathbf{g}} = [\hat{g}_1, \hat{g}_2]^T$ determined by such an approximation is known as Stochastic Gradient Descent and comes with at least two key advantages related to accelerated learning and sensitivity to getting trapped in local minima:

1. Instead of updating the weights after having presented all examples to the network, they are updated after the presentation of each mini-batch. This accelerates the learning speed.
2. Instead of getting trapped in the closest local minimum of the objective function, the random direction suggested by each mini-batch is only known to point towards the minimum on average. This means that the steps are not deterministic and not always in the direction of the negative gradient of the objective function. This often helps avoiding small local minima.

Mini-batch: In the simple example above the stochastic approximation of the total sum was based on using only one of the terms each time you would like to make an approximate Gradient Descent step. Another alternative is to use a subset of terms for each update. Such a subset is known as a *mini-batch*.

Epoch: In this context the term *epoch* is also often employed. This refers to *a complete cycle through the whole set of training examples*. Thus if the number of training examples is 100 and the mini batch size is 10, then one epoch is completed after 10 consecutive mini batches have been used to take 10 approximate Gradient Descent steps. Notably, for each epoch, every training example is only part of one of the mini-batches (subsets). Moreover, one should note that if the number of training examples and the desired size of the mini-batches are not compatible (for example if there are 97 examples and the desired batch size is 10), then one of the mini-batches will be smaller than the rest.

Adam: Adam is an adaptive stochastic gradient method. Many different algorithms have been proposed to automatically adjust the size and direction of the steps taken during Stochastic Gradient Descent search. The main problems are to avoid oscillations in the search direction from one iteration to the next and to ensure convergence close to a local minimum without just hovering around it. One of the most popular algorithms proposed in 2015 to solve this problem is called *Adam* and is available in both PyTorch and Tensorflow2. Remark: The name Adam is derived from “adaptive moment estimation”, which has to do with estimating how much the search direction is oscillating.

Problem 3.5: Fast weight updating

Assume your training set consists of $N = 64 \cdot 10^6$ examples and that your mini-batch size is 64, meaning you have in total 10^6 mini-batches (subsets) per epoch. After having presented all the training set 3 times to the network during 3 epochs, how many times have the network weights been updated if one is employing conventional Gradient Descent and Stochastic Gradient Descent, respectively? Why does this result suggest that Stochastic Gradient Descent is more attractive to be used?

Problem 3.6: Reduced problems with local minima

Why does Gradient Descent have difficulties in terms of finding the global optimum when the objective function has a lot of shallow local minima? For what reason does Stochastic Gradient Descent have a reduced risk of getting stuck in small/minor local minima (when they exist)?

4 Theory II: Building and training ANNs in PyTorch

In the following we will go through, step by step, how to arrive at a general workflow for training an ANN on a dataset using mini-batch Stochastic Gradient Decent with performance monitoring during the training. For a first brief overview, see the following list of steps:

1. Converting raw data into convenient PyTorch **Dataset** objects or downloading standard common datasets.
2. Splitting the original dataset into three separate sets for ‘training’, ‘validation’, and ‘validation’, respectively.
3. Using PyTorch **DataLoader** to make life easier when doing mini-batch Stochastic Gradient Decent.
4. Defining, designing and initiating ANN architectures using **nn.Sequential(...)** available from the submodule **torch.nn**, which contains useful predefined ANN functionality and building blocks.
5. Picking a suitable built-in PyTorch cost function.
6. Specifying and using PyTorch built-in optimizers that automatically take care of the Stochastic Gradient Decent update step for all parameters in our model. This is where we provide this optimizer object together with the parameters of our model and specify the learning rate.
7. Define and implement a *model performance evaluation* criterion. This means choosing which performance metric to use for quantifying the performance of your model after/during training. For regression, the performance measure used here is typically the same as the one used for the training (parameter fitting). However, for classification problem, the cost function used for training (parameter fitting) is actually not the number of classification errors, but instead an indirect measure. Therefore, the model performance evaluation (typically the number of classification errors) is different than the objective function used for training.
8. Implement the training loop where the actual iterative training/update of your ANN model happens. This step corresponds exactly to the Gradient Decent update loop you are familiar with from before, this time using Stochastic Gradient Descent. Here we also specify the number of epochs (total number of iterations through the whole dataset), as well as how often we will perform model evaluation using all examples present in both the training and validation sets. More specifically, this step consists of a double loop:
 - Inner loop: For each epoch, iterate through a sequence of single steps of mini-batch Stochastic Gradient Decent parameter updates. This practically boils down to doing a single ordinary Gradient Decent update of the model parameters while using a stochastic cost defined using only a small randomly selected subset of the training examples.
 - Outer loop: Embed the inner loop in an outer loop iterating over the desired/maximum number of epochs specified. After each block of a pre-defined number of completed epochs, make a model evaluation using all examples present in both the training set as well as in the validation set.
9. Evaluate the final trained model on the independent external test dataset to arrive at an objective (unbiased) estimate of its real (future) performance in terms of our chosen metrics.

In the following pages, will go through how to implement each of the above steps in PyTorch. In order for you to have a preliminary mental model of how all the above steps translate into code and relate to each-other, the next page contains the corresponding pseudo-code (still without details - to be filled in as we proceed). In each of the following sections, we will focus on PyTorch implementation and details of each of the steps in the workflow template. At the end we put everything together and give a more implementation detailed version of the same workflow.

```
1 # CODE TEMPLATE FOR TRAINING NEURAL NETWORKS IN PYTORCH - Conceptual / No-code
2 import torch
3 from torch import nn
4 from torch.utils.data import DataLoader, TensorDataset,
5 from torchvision import datasets
6 from torchvision.transforms import ToTensor,
7
8 # 1. & 2. Split or get Data into train, validation and test sets
9
10 # 3. Make a DataLoader object to take of 'serving' you mini-batches of your data.
11
12 # 4. Define and construct your neural network architecture.
13
14 # 5. Define and Pick a suitable cost function depending on your problem type
15 # regression or classification problem
16
17 # 6. Define and Set an Optimizer object that takes care of
18 # the Stochastic Gradient Decent update step.
19 # Specify also the model parameters and learning rate
20
21 # 7. Define how you evaluate your model performance as a function.
22
23 # 8. Set number of epochs to train for
24 # Set the epoch interval model performance evaluation, ie evaluate model every k'th epoch.
25 # NN Training
26 #
27 #Outer loop: Iterate through whole dataset nr_epochs times and evaluate repeatedly
28 for epoch_i in range(nr_epochs):
29     # Model Performance evaluation block
30     if epoch_i%eval_every_kth ==0: # Eval model very k'th epoch.
31         # set model into evaluate mode #
32         # EVALUATE Model 'performance' on whole train and validation dataset
33         # reset model into train mode.
34         # track performance measures from both train and val sets
35
36         # Optional stopping criteria based on validation and/or
37         # train set performance/cost to avoid overfitting.
38
39     # Inner loop: Model Parameter update using mini-batch SGD blocks
40     for X_batch, y_batch in my_dataloader: # Iterate through mini-batches
41         # Make Prediction on mini-batch
42
43         # Compute cost for mini-batch
44
45         # Set the grads of all model params to zero.
46
47         # compute and populate gradients of model params
48
49         # Instruct optimizer to take one update step (using SGD)
50
51 # 9. Evaluate resulting 'performance'
```

4.1 The PyTorch object Dataset

Raw collected data can be stored inside a computer file and storage system in very many ways and in very many formats. However, when we want to feed our inputs $\mathbf{x}_n$ into our ANN models built in PyTorch, then these inputs must be already in the form of torch tensors. This is where the PyTorch object `Dataset` object comes into play. Its job is to deal with accessing and pre-processing single data items of a dataset stored in memory (hard-drive) and returns it as a PyTorch tensor. For a starter you are expected to read the following:

https://pytorch.org/tutorials/beginner/basics/data_tutorial.html#datasets-dataloaders

As will be discussed below, when we need to create mini-batches (small subsets of the dataset) of data items, this will be done with a PyTorch `DataLoader` object, which uses your dataset object iteratively.

NOTE: In practice, we will never use a dataset object explicitly, but always access data by being served mini-batches by a `DataLoader`.

4.2 Already prepared/preloaded PyTorch Datasets

Many image datasets commonly used for education and competitive comparisons of different algorithms are available for automatic downloading via the sub-module `datasets` inside the package `torchvision`. This package is a dedicated ANN Library in PyTorch for everything related to image processing and computer vision, such as functionality for getting image/video datasets, torch mathematical operations common for images etc.

Example: In a later assignment you will use the provided dataset called FashionMNIST. This is a large collection of 60000 small (28x28 pixel) images of clothing items, each with an assigned label from 10 different classes of clothing. Here follows a code snippet that accesses the FashionMNIST dataset using the PyTorch framework.

```
1  ## Code to access the FashionMNIST
2  import torch
3  from torch import nn
4  from torch.utils.data import DataLoader
5  from torchvision import datasets
6
7  from torchvision.transforms import ToTensor, Lambda, Compose
8  import matplotlib.pyplot as plt
9
10
11 # Download training data from open datasets.
12 training_data = datasets.FashionMNIST(
13     root="data",
14     train=True,
15     download=True,
16     transform=ToTensor(),
17 )
18
19 # Download test data from open datasets.
20 test_data = datasets.FashionMNIST(
21     root="data",
22     train=False,
23     download=True,
24     transform=ToTensor(),
25 )
```

Here the argument `download` indicates if we want to download the dataset to our local storage or not. If we have already downloaded this dataset before, then we set this to `False`, else to `True`. If set to `True`, then in the argument `root` we specify the relative path and folder name to download the dataset into. In this case it is the folder “data”, which PyTorch will create for us.

Some datasets come with already pre-selected train and test datasets. Moreover, the argument `train` simply specifies which dataset we want for our dataset object. In the above code snippet, we create two dataset objects, one for the training set and one for the test set (which contains 10000 images together with their labels). Finally, the argument `transform` is used to specify that we want all the data to be returned to us in torch tensors. For images we specify the function `ToTensor()`

that transforms images into proper torch tensors.

Remark.

For a more thorough explanation of what each argument means and represents, see the official PyTorch tutorial: https://pytorch.org/tutorials/beginner/basics/data_tutorial.html.

4.3 Splitting your data into 'train-', 'validation-' and 'test-' -sets

The PyTorch call

```
torch.utils.data.random_split(dataset, [train_size, val_size, test_size])
```

helps to randomly split the original dataset into 3 parts denoted as the training set, the validation set, and the test set, respectively. As suggested by the names, these sets have different roles:

- *The training set* is used for parameter fitting, meaning minimizing an objective function.
- *The validation set* is used to tune/select hyperparameters, like the initial learning rate η and the ANN architecture (number of layers, number of nodes per layers, activation functions). A second use of the validation set is for early stopping, meaning another way of terminating the iterative numerical optimization, in order to avoid overfitting.
- *The test set* is used to obtain an independent and therefore unbiased performance estimate for the final prediction model selected by using both the training and validation sets.

The following code snippet shows an example of how to construct the three sets from your original dataset object.

```
1 # TEMPLATE: SPLITTING YOUR DATA INTO 'TRAIN-', 'VAL-' AND 'TEST-' sets.
2 original_data = # Init your PyTorch Dataset object
3 size_of_original_data = len(original_data)
4
5 # Specify split fractions: !Must sum to 1!
6 train_fraction = 0.70
7 val_fraction = 0.10
8 test_fraction = 0.20 # for show purposes
9
10 # Determine size of each set
11 train_dataset_size = int(train_fraction * size_of_original_data)
12 val_dataset_size = int(val_fraction * size_of_original_data)
13 test_dataset_size = int(size_of_original_data - train_dataset_size - val_dataset_size)
14
15 # Split whole original data into train, val and test datasets
16 train_dataset, val_dataset, test_dataset = torch.utils.data.random_split(original_data,
17                                                                           [train_dataset_size,
18                                                                           val_dataset_size,
19                                                                           test_dataset_size])
20
21 # Sanity checking
22 print(f" Train set Size: {len(train_dataset)}")
23 print(f" Val set Size: {len(val_dataset)}")
24 print(f" Test set Size: {len(test_dataset)}")
```

Remark.

For a more detailed exploration of the need for validation and test sets and its relation to the concepts of *over-fitting* and *generalization*, see Section 5.5.3 (p. 131) in the freely available book *Deep learning with PyTorch* by Stevens, Antiga and Viehmann:

<https://pytorch.org/assets/deep-learning/Deep-Learning-with-PyTorch.pdf>

4.4 Define and construct fully connected FANN models in PyTorch

In this section you will learn or become reminded how to define *custom architectures* of ANNs in PyTorch. NOTE: Most of the remaining of this section you might have seen in a previous assignment. If this is the case, just use the following as a good reminder.

4.4.1 Specify your FANN architecture:

You will be defining your FANN architecture using the model constructor `nn.Sequential(...)` by specifying all the features of the network that your data will flow through. These features are mainly the number of layers, the number of nodes (neurons) in each layer, and the activation functions of these nodes.

4.4.2 Parameter initialization

Every time you initialize a new FANN model in PyTorch, that model will automatically (outside your control) have its parameters to be fitted (weights and biases) set to fresh randomly selected small values.

4.4.3 Example 1: A FANN with 2 inputs, 2 hidden layers and 1 output

For an illustration, see Fig.8. The activation function *ReLU* should be used for the hidden layer neurons and the activation function *tanh* should be used at the output so that network output values are restricted to the interval $[-1, 1]$.

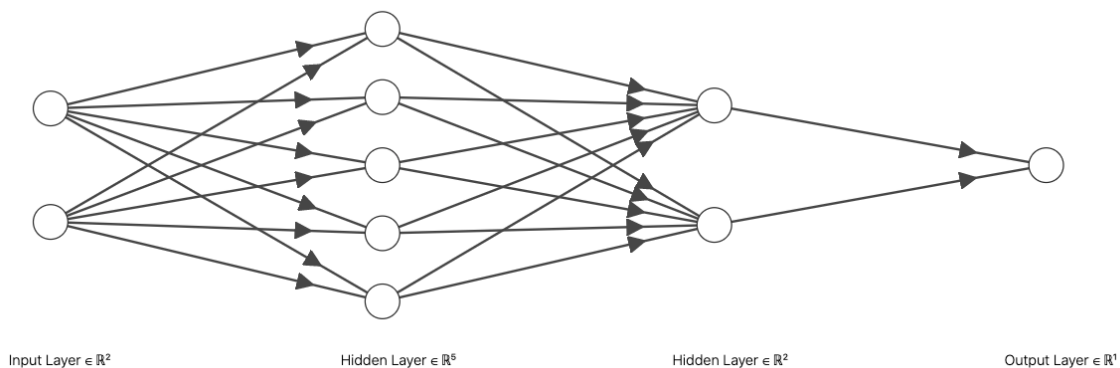


Figure 8: A FANN with 2 inputs, 2 hidden layers and 1 output.

The code needed to create this network in PyTorch is presented here:

```
1 # Simple way to define a Fully connected feed forward ANN with two inputs, 5
2 # neurons in the first hidden layer, 2 neurons in the second hidden layer
3 # and a single output neuron. NOTE: Different types of activation functions
4 # in each layer
5 model = nn.Sequential(
6     nn.Linear(in_features=2, out_features=5),
7     nn.ReLU(),
8     nn.Linear(in_features=5, out_features=2),
9     nn.ReLU(),
10    nn.Linear(in_features=2, out_features=1)
11    nn.Tanh()
12 )
```

4.4.4 Example 2: A FANN with 3 inputs, 2 hidden layers and one output

For an illustration, see Fig.9. Here the ReLU activation function should be used for all hidden neurons and no activation function should be employed for the output neuron.

The code needed to create this network in PyTorch is presented here:

```
1 # Simple way to define a Fully connected feed forward ANN
2 model = nn.Sequential(
3     nn.Linear(in_features=3, out_features=5),
4     nn.ReLU(),
5     nn.Linear(in_features=5, out_features=5),
6     nn.ReLU(),
7     nn.Linear(in_features=5, out_features=1)
8 )
```

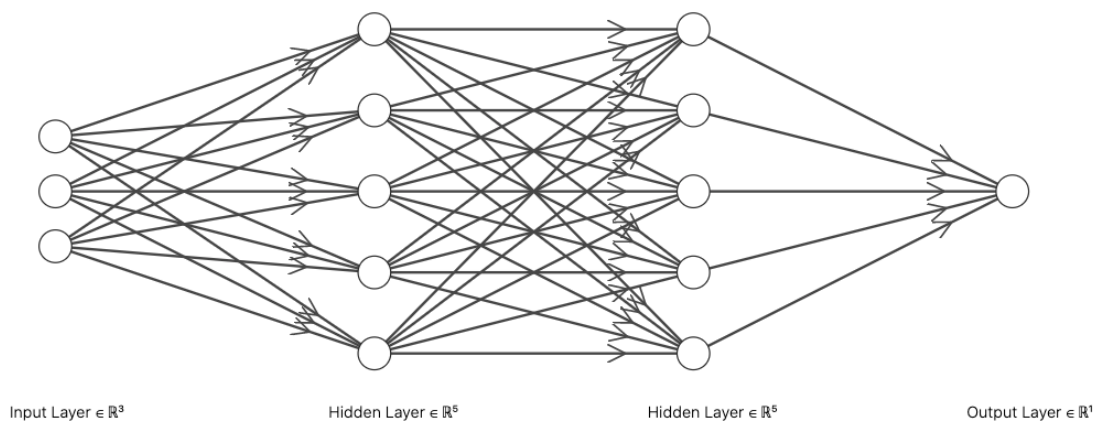


Figure 9: A FANN with 3 inputs, 2 equally large hidden layers and one output.

4.5 How to feed data into your ANN models to make predictions

Once an ANN model has been specified and has been instantiated with some parameter values, it is ready to be used to make predictions using old and new data. In PyTorch this is easily done as follows, feeding in either a single data point $\mathbf{x}_n$, or a set of points (provided that each data point corresponds to one row in a two-dimensional tensor) is as simple as shown in the following code snippet.

```
1 model = nn.Sequential(  
2     # Model architecture components  
3 ):  
4  
5 X = # tensor object where each 'row' (first index of tensor) corresponds to a single data-point  
6 y_preds = model(X)
```

As perhaps already seen in a previous assignment, taking data from our `Dataset` object in order to feed it into our ANN model during training. is done with PyTorch's `DataLoader` object, which will be the topic of the next section.

4.6 Simplified data management using “DataLoader”

PyTorch makes it easy to perform Stochastic Gradient Descent based training of any FANN architecture without having to go into great details. Here follows a code snippet illustrating how the management of your datasets can be simplified using `DataLoader`.

```
1 X_data = # all your x_n's in tensor format  
2 y_data = # all your y_n's in tensor format  
3  
4 # making A PyTorch dataset object for your data  
5 my_dataset = TensorDataset(X_data, y_data)  
6  
7 # making a PyTorch dataloader object  
8 my_dataloader = DataLoader(my_dataset, batch_size=50, shuffle=True, )  
9  
10 for epoch_i in range(nr_epochs): # epoch counter  
11     for X_batch, y_batch in my_dataloader:  
12         # Train network on a single mini_batch  
13         # ie. perform gradient decent on a single data mini batch  
14         # Every iteration of this inner loop will give you a  
15         # mini-batch subset of the original data, presented in X_batch and y_batch
```

4.7 Simplified updating in PyTorch using “optimizer”

As mentioned above, PyTorch makes it easy to perform Stochastic Gradient Descent based training of any FANN architecture without having to go into as much details as before. For the parameter updating, there is an object `optimizer` that takes care of updating each and every parameter in our model. We only need to feed into this optimizer with all parameters defining our ANN architecture,

and also specify some other optimization specific arguments like the value of the learning-rate η discussed in previous assignments.

```
1 # Mini-batch SGD with PyTorch optimizer in the train loop
2 model = # Define and construct your model
3
4 # Pick a Cost function
5 cost_function = # Pick your PyTorch cost/loss function from torch.nn
6
7 # Define and Set an optimizer object.
8 # Input as arguments the collection of parameters of your model
9 # and your desired learning rate aka eta..
10 optim = torch.optim.SGD(model.parameters(), lr=... )#
11
12 for epoch_i in range(nr_epochs): # epoch counter
13     for X_batch, y_batch in my_dataloader:
14         y_preds = model(X_batch) # Make Prediction on batch
15         cost = cost_function(y_preds, y_batch) # Compute cost
16         optim.zero_grad() # zero the grads of all model params
17         cost.backward() # compute J gradient of all model params
18         optim.step() # take one update step for all model params
```

As you can see in the code snippet provided here, in PyTorch there is a tight built-in (hidden) relationship between the parameters of the ANN model, the optimizer, and the cost function. After you perform auto-differentiation by calling `cost.backward()`, the optimizer is able to fetch the gradient values created and use them for updating.

A simplifying picture regarding how these three things relate to each other is provided for example in the first edition of the book *Deep Learning with PyTorch* by Stevens, Antiga and Viehmann (section 5.5.2, p.128), see course home page or the following link:

https://isip.piconepress.com/courses/temple/ece_4822/resources/books/Deep-Learning-with-PyTorch.pdf

4.8 Model performance evaluation - during and after training

The following code snippet shows an example of how you can implement a model performance evaluation that takes a model of interest and a dataset as inputs and then outputs an performance estimate (in this case the mean squared error - MSE) of the model.

The performance measure used for regression problems is usually the same during training and validation. Thus, if the mean squared error (MSE) is used as the cost function during training, then one is usually also interested in this quantify for the final trained model. However, as also already mentioned, for improved human interpretability often the RMSE value is reported instead of the MSE value.

For classification problems the situation is quite different compared to regression problems. This is because using the the number of classification errors as the cost function will not be useful when training by means of variants of Gradient Descent. This is because this cost function is constant almost everywhere. As a consequence, the gradient of this cost function is zero almost everywhere! Therefore, another cost function must be employed for the gradient based training. Such cost functions will not be discussed any further here, as the focus now is on regression problems.

Note: When we want to obtain a performance estimate, we should not perform any Gradient calculations and we should have only one single batch - meaning the whole dataset that we want to evaluate model performance on.

```
1 from sklearn.metrics import accuracy_score
2
3 def evaluate_model_performance(eval_dataset, model):
4     # Function that takes in a model and a dataset
5     # and outputs an performance estimate of the model in the form of the MSE value
6
7
8     # Make a Dataloader for the dataset.
9     # Note, we are not performing any SGD here, so our batch
10    # size is the whole # dataset, for which we want to evaluate
11    # model performance on.
12    d_loader = DataLoader(dataset = eval_dataset, batch_size=len(eval_dataset))
13
14    cost_function = nn.mse() # For regression
15
```

```
16 # Make predictions for the whole eval dataset in one iteration
17 model.eval() # Sets model in evaluation (inference) mode
18 # Dropout, layer normalization etc are turned off
19 with torch.no_grad():
20     for X, y in d_loader:
21         raw_y_preds = model(X)
22 model.train() # Reset the model in "training" mode
23 # Dropout and layer normalization etc are turned on
24
25 # Calculate the MSE
26 eval_mse= cost_function(raw_y_preds, y).item()
27
28
29 return eval_mse
```

4.9 Putting it all together: The flow of ANN training

Here follows an example of a complete code snippet where all pieces are put together to get a workflow for ANN training.

```
1 # CODE TEMPLATE FOR TRAINING NEURAL NETWORKS IN PYTORCH
2 import torch
3 from torch import nn
4 from torch.utils.data import DataLoader, TensorDataset,
5 from torchvision import datasets
6 from torchvision.transforms import ToTensor,
7
8 # Split or get Data into train, validation and test sets
9 train_dataset, val_dataset, test_dataset # Split or Get these three datasets
10
11 # Make a DataLoader object to take of of mini-batches for data.
12 train_dataloader = DataLoader(dataset = train_dataset, batch_size = ..., shuffle = True )
13
14 model = # Define and construct your neural network architecture.
15
16 # Pick a Cost function
17 cost_function = # Pick your PyTorch cost/loss function from torch.nn
18
19 # Define and Set an Optimizer object
20 optim = # Some choice of optimizer and specify model params and learning rate lr
21
22 # Define how you evaluate your model performance
23 def evaluate_model_performance(model, dataset, ...):
24     # Function that computes performance measures of your model
25     # NOTE: Don not keep track of grads when evaluating model.
26     return metric_1, metric_2, ...
27
28 for epoch_i in range(nr_epochs): # Iterate through whole dataset nr_epochs times
29
30     if epoch_i%eval_every_kth ==0: # Eval model very k'th epoch.
31         model.eval() # set model into evaluate mode #
32         # EVALUATE Model 'performance' on whole train and validation dataset
33         train_cost, train_acc, ...= evaluate_model_performance(model=model, dataset = train_set)
34         val_cost, val_acc, ... = evaluate_model_performance(model=model, dataset = val_set)
35         model.train() # reset model into train mode.
36         # track performance measures from both train and val sets
37
38         # Optional stopping criteria based on validation and/or
39         # train set performance/cost to ovoid overfitting.
40
41     for X_batch, y_batch in my_dataloader: # Iterate through a batch
42         y_preds = model(X_batch) # Make Prediction on mini-batch
43
44         cost = cost_function(y_preds, y_batch)# Compute cost for mini-batch
45
46         optim.zero_grad() # Set the grads of all model params to zero.
47
48         cost.backward() # compute and populate gradients of model params
49
50         optim.step() # instruct optimizer to take one update step (using SGD)
51
52 # EVALUATE Model 'performance' on whole train and Validation AND TEST dataset
53 test_cost, test_acc, ...= evaluate_model_performance(model=model, dataset = test_set)
```

5 Task B: Non-linear univariate regression using FANNs

The task here is to use the dataset `MarvinMinsky.csv` to determine how well you can make a small/simple FANN to model a non-linear functional relationship between some input x and the output y . Here follows code showing how to import the dataset (which is available on the course home page).

```
1 from numpy import genfromtxt # we use this to load data from a csv into a numpy array
2 my_data = genfromtxt('dataset_Marvin_Minsky.csv', delimiter=',', skip_header=1) #both x and y are
   here
3
4 X = my_data[:,0] #Extract the x_n
5 y_obs = my_data[:,1] #Extract the y_n
6
7 # comment about each row being a data-pair # reshape to make each row in X a data-point x_n
8 X = torch.tensor(X).reshape(len(X),1).float()
9 # turn to column vector, each row corresponds to each y_n
10 y_obs = torch.tensor(y_obs).reshape(len(y_obs),1).float()
11
12 my_dataset = TensorDataset(X, y_obs) # create dataset by turning tensors into datasets objects
13 my_dataloader = DataLoader(my_dataset, batch_size=..., shuffle=...) # create your dataloader
   from dataset
```

Since this is about a non-linear functional relationship, you will need to use a more flexible/powerful ANN architecture than in the previous assignment where the relationship was linear (plus noise). Try a network with structure 1-3-3-1, and increase the number of hidden nodes if needed. As node (neuron) activation function in the hidden layers, use `tanh()`. The goal of this sub-task is to arrive at a trained model that makes predictions similar to or better than the ones in Fig. 10.

Remark

You may also try with the activation function `ReLU` instead of `tanh()`. However, our previous experience is that this choice creates more problems with local minima! Thus this is an example showing that `ReLU` is not always the best choice for regression problems.

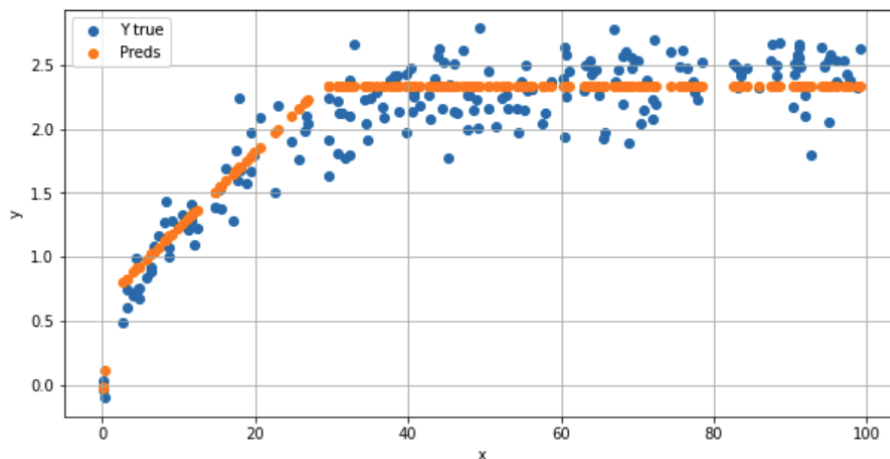


Figure 10: Example of trained ANN predicting the Marvin Minsky data.

You should produce a plot showing the cost as a function of iterations. You should also produce the final value of the cost function (MSE) at the end of the training, which should be below 0.1.

6 Task C: Approximate a bell-shaped bivariate function

Here we will continue with relatively simple and small FANN models but for a bit more challenging 2-variable regression problem, meaning $\mathbf{x}_n \in \mathbb{R}^2$. The specific task is to fit an ANN to a dataset that when plotted looks to come from a bell shaped surface (with noise), for one example see Fig. 11.

More specifically, the task is to fit the parameters of a few of FANN architectures to a dataset $D = \{(\mathbf{x}_n, y_n)\}$ where each 2-dimensional input $\mathbf{x}_n$ is drawn from a uniform distribution on the square $[-5, 5] \times [-5, 5]$ and the response values (our y_n) are determined as

$$y(\mathbf{x}) = e^{-\frac{\mathbf{x}^T \mathbf{x}}{16}} = e^{-\|\mathbf{x}\|^2/16}. \quad (3)$$

Thus the task to train a FANN so that for a new input $\mathbf{x}_n = [x_1, x_2]$, it will output a prediction of $\hat{y}_n$ very close to the desired value.

One possible interpretation of the dataset studied here: Assume that this bell shaped function reflects the yield of a manufacturing process as a function of two variables such as temperature and pressure. Also assume that the temperature and pressure varies over time and that the dataset

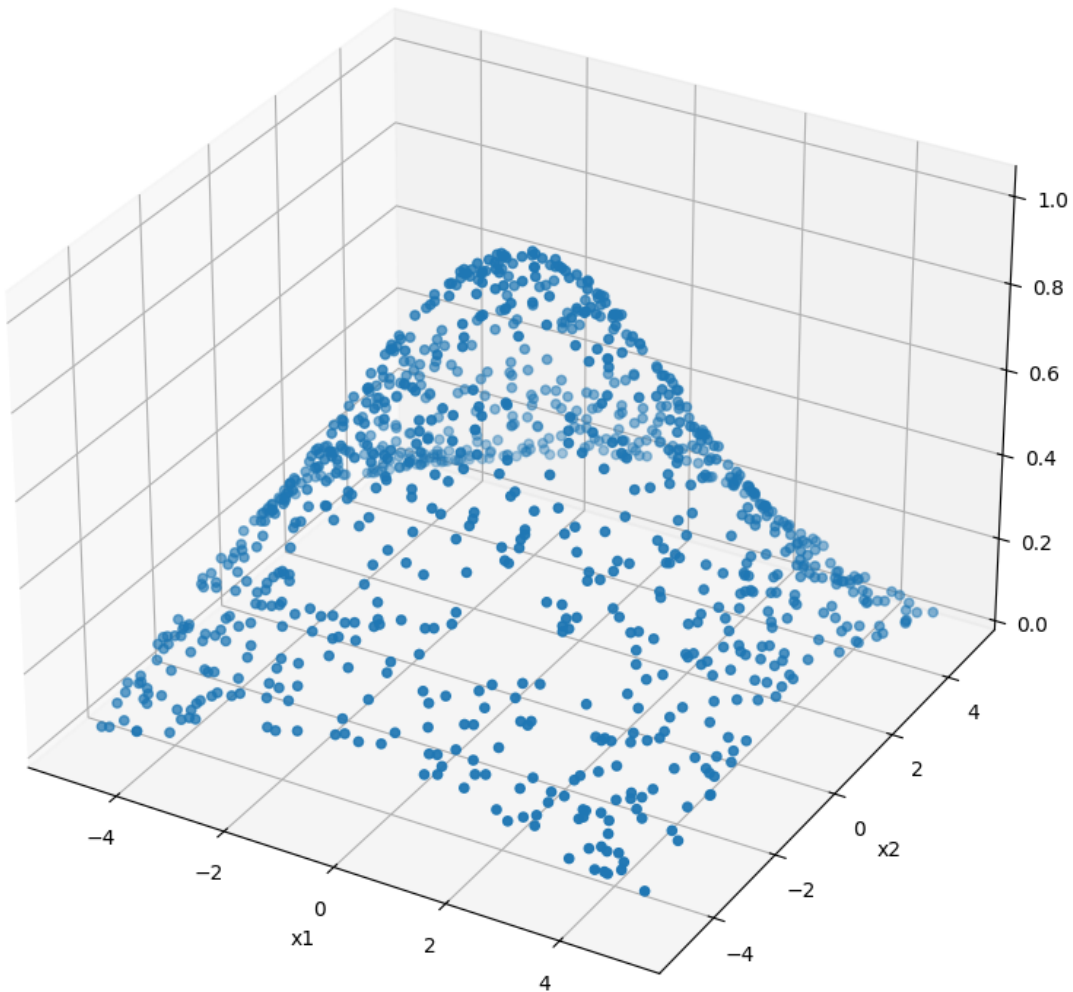


Figure 11: 3D scatter plot of the data-points from the bell shaped function $y(\mathbf{x}) = e^{-\|\mathbf{x}\|^2/16}$.

available here is the result of collecting data from the process at regular time points t_n over the last months. Finally, assume that the variable $x_1(n)$ for the n :th example equals a normalized temperature, meaning $x_1(n) = T(n)/s_T$ where $T(n)$ is the true temperature and s_T is the standard deviation across all temperatures in the dataset. Similarly, assume $x_2(n) = P(n)/s_P$ where $P(n)$ is the pressure value for sample n .

No validation and test set: Notably, we will not yet consider splitting our training set into train-, validation and test sets for model performance. The idea again is to focus only on getting good results on the training set.

Selection of model family: The visualization of the dataset as in Fig. 11 indeed suggests that it should be quite successful to fit a parameterized bell shaped function to these data. However, here we will instead consider the possibility to make relatively small FANNs fit these examples. The reasons for using FANNs instead of a bell shaped function are at least three: (i) In most cases we have too few examples in the dataset to clearly see what is the shape of the underlying function. In such cases it is reasonable to use a quite general/flexible FANN with capacity to fit not only bell shaped functions but also other functions. (ii) When there are more than 2 input variables, we cannot make visualizations of this kind to guide the selection of model family. Therefore, using FANNs is again a reasonable choice as they can model arbitrary complex functions if they are selected to be large enough. (iii) In this assignment/course, the main goal is to understand more about how to fit FANNs to data.

(a) Generate $N=1000$ noisy data-points from the above bell-curve function with the following code where the standard deviation of the zero mean additive Gaussian noise equals `sigmaNoise=0.04`. Next you create a dataset object and a data-loader object ready to be used for the desired ANN training.

```
1 N = ..... # Nr of data-points to sample and generate
2 x = 10*torch.rand(size=[N,2]) - 5
3 mean_true = torch.tensor([0.0, 0.0])
4 sigma_noise=0.04
5 y = torch.zeros(N).reshape(-1,1)
6 for i in range(len(x)):
7     y[i] = torch.exp(-(x[i,:]-mean_true).T@(x[i,:]-mean_true))/16) + sigma_noise*torch.randn(1)
8
9 # Plotting
10 fig = plt.figure()
11 fig.set_figheight(10)
12 fig.set_figwidth(10)
13 ax=plt.axes(projection='3d')
14 ax.scatter(x[:,0], x[:,1], y)
15 ax.set_xlabel('x1')
16 ax.set_ylabel('x2')
17
18 my_dataset = TensorDataset( ..... ) # do
19 batch_size = .... # do
20 my_dataloader = DataLoader( ..... ) # do...
```

Before continuing, you should make sure that your plot of your synthetically generated data points looks similar to Fig. 11 (now also having noise).

(b) Design the following 2 different FANN architectures in your Python code:

Architecture I: 2-40-1 with ReLU activation for all neurons in the single hidden layer.

Architecture II: 2-40-40-40-1 with ReLU activation for all neurons in the 3 hidden layers.

(c) For each of the above ANN architectures, build these in PyTorch and train each of them to the best of your abilities in order to achieve the smallest cost. You may have to try different values for the following two hyper-parameters to get good results:

- The learning rate parameter η should be adjusted according to the architecture used. If you find your training diverging towards infinity, try with a smaller value. Tip: Start with $\eta = 0.01$.

- Put the maximum number of epochs to 400 for a starter. But this number should be adjusted if you find that your training cost curve (it is always good to monitor how the cost changes as a function of iterations) is still going down.

(d) Prepare documentation containing at least the following information:

1. your chosen learning rate
2. training cost curves as a function of the number of mini-batches
3. the final cost function value
4. a 3D plot showing the surface generated by the trained ANN when you feed into it the same domain region $[-5, 5] \times [-5, 5]$, meaning similar to Fig.12. The code snippet below will be useful for this task.

```
1 x1 = torch.linspace(start=-5, end=5, steps=100)
2 x2 = torch.linspace(start=-5, end=5, steps=100)
3 X, Y = torch.meshgrid(x1, x2)
4
5 s = torch.stack([X.ravel(), Y.ravel()]).T # all grid coordinates
6
7 y_true = torch.zeros(len(s)).reshape(-1,1)
8 y_preds = torch.zeros(len(s)).reshape(-1,1)
9 for i in range(len(s)):
10     y_true[i] = torch.exp(-((s[i,:]-mean_true)@((s[i,:]-mean_true).T))/16) + 0.0*torch.randn(1)
11     with torch.no_grad():
12         y_preds[i] = model(s[i,:])
13
14 #3D Surface plotting
15 fig, (ax1, ax2, ax3) = plt.subplots(1, 3, subplot_kw={'projection': '3d'}, figsize=(12, 5))
16
17
18 ax1.scatter(s[:,0], s[:,1], y_true, c='blue', label='truth', alpha=0.9)
19 ax1.set_title('Truth')
20 ax1.set_xlabel('x1')
21 ax1.set_ylabel('x2')
22 ax1.legend()
23 ax1.set_zlim(-0.1, 1.2) # Set z-axis limits
24
25 ax2.scatter(s[:,0], s[:,1], y_preds, c='orange', label='Fitted FANN', alpha=0.9)
26 ax2.set_title('Fitted FANN')
27 ax2.set_xlabel('x1')
28 ax2.set_ylabel('x2')
29 ax2.legend()
30 ax2.set_zlim(-0.1, 1.2) # Set z-axis limits
31
32 d=y_preds-y_true
33 print("abs max=", np.max(np.abs(d.numpy())))
34 ax3.scatter(s[:,0], s[:,1], d, c='red', label='Difference', alpha=0.9)
35 ax3.set_title('Difference')
36 ax3.set_xlabel('x1')
37 ax3.set_ylabel('x2')
38 ax3.legend()
39 ax3.set_zlim(-0.1, 1.2); # Set z-axis limits
```

7 Task D: Model selection using a validation set

In the previous task you just trained two FANN models using all examples in your training set. In order to estimate their true performance on independent data, we need to have such data collected in a validation set.

(a) Generate five new random noisy test examples using the true model as your validation set, and estimate the performance of the final two FANNs using only these 10 examples. Which of the two FANNs has the lowest estimated RMSE?

(b) Now consider the FANN in (a) that offered the lowest RMSE: Notably, this is the FANN you would have chosen if you had to choose based on the result from estimating performance using the 10 examples in the validation test. Determine the (almost) true performance of this selected FANN by calculating the RMSE when using $N_{\text{BIG_TEST}} = 25000$ new test examples generated in the same way as before. Thus you should generate 25000 new test examples using the same generative model as before, and then calculate the RMSE for the most promising model according to the validation set in (a). Finally, explain why this testing with 25000 independent test examples will give a value very close to the true (but unknown) RMSE for this FANN?

(c) Compare the performance estimate obtained in (a) using the 10 validation examples with the (almost) true performance obtained in (b) using 25000 test examples. Is the estimate from (a) optimistic or pessimistic compared to the true performance? Do you think the number of validation examples should have been more than five?

(d) Also determine a good estimate of the true performance of for the network architecture that was rejected based on the comparison using the five test examples. What true performance does this network have? Is this performance better or worse than the network that was selected?

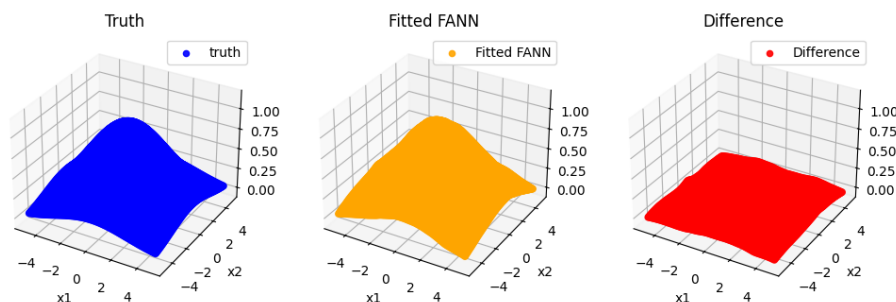


Figure 12: Example results where a large FANN has been successfully fitted to 1000 data generated from a bell-shaped function. The subplot to the left shows the true function, the center plot shows the response surface of the FANN, and the right subplot shows the difference between the FANN and the true function, showing relatively small errors according to the z-axis.

8 Task E: Early Stopping Using a Validation Set

For one of the FANN architectures from above (your choice), repeat the previous exercise but using a smaller dataset and using early stopping as follows.

1. Reduce the dataset or generate a new one with in total 100 examples. Split this original dataset into 50 examples for training, 25 examples for validation, and 25 examples for testing.
2. After completing each epoch i : Determine an estimate $\hat{r}_i$ of the unknown network performance (RMSE) by means of the 25 examples in the the validation set.
3. Starting after ending the 6:th epoch: After completing very single epoch, study the latest six performance (RMSE) estimates $\hat{r}_1, \hat{r}_1, \dots, \hat{r}_6$, where $\hat{r}_6$ corresponds to the most recent estimate. Compare the average of the performance estimates $\hat{r}_i$ obtained during the last 3 epochs (4-6) with the corresponding average obtained during the three earlier epochs (1-3). If the more recent average corresponds to a worse performance, then stop the training, otherwise continue with one more epoch.
4. After termination, print which at which epoch the termination occurred. Then estimate the performance (RMSE) of the resulting model using the 25 test examples in the test set. Also make a 3D graph showing the surface created by this final model.
5. After termination, also estimate the performance (RMSE) of the resulting final model using a big test set consisting of $N_{\text{BIG_TEST}} = 25000$ independent new test examples, which will get you very close to the true performance. How well does the RMSE value based on the 25 test examples agree with the (almost) true RMSE value obtained when testing with 25000 test examples?
6. Repeat the task 3-4 times and write down the results. Compare them and try to reflect how much variability there is for data sets of the same size, generated by the same underlying model.

9 DEADLINE AND EXAMINATION

This examination will consists of the following parts:

(1) Answers and Results: You should upload to Studium your answers and results to all mandatory problems present above as one pdf. **Deadline: See course information..**

NOTE: If you are working in pairs, each person has to submit their own document, even if it is almost identical. You are of course always responsible for your own submission and must be able to explain all of it during seminars and similar examinations.

(2) Python code as one notebook or as wrapped inside a zip file: The code should be ready to run in Google Colab. If your code consists of more than one notebook (and/or some additional code called from inside the notebook) your instructions how to run the code should be provided as a text file called `readme.txt`. Make sure that all files you are submitting have your first name and surname as part of the file name, for example like:

`Assignment_2_reflections_Mats_Gustafsson.pdf`

NOTE: If you are working in pairs, each person has to submit their own code, even if it is almost identical. Notably, you are of course responsible for all your the code your submit, and you must be prepared to discuss and explain it during the seminars.

(3) Personal reflections and feedback: An uploaded pdf document to Studium providing personal reflection and feedback: This pdf document should contain reflective answers to the following questions:

(a) In this assignment you have tried to train a FANN using a dataset generated from a a bi-variate with a bell shaped response surface. Why does it seems reasonable/possible to use a FANN to model/approximate such a function?

(b) Why is it always dangerous to train without having a validation dataset used for termination of the training (early stopping), and/or for selection between different already trained prediction model/network alternatives?

(c) Why is it difficult beforehand to know how simple/complex (for example number of layers) your FANN model should be in order to be able to approximate the underlying function of interest in sufficient detail?

(d) Why is it difficult to know how well a particular FANN model, which has a particular architecture with particular activation functions, can learn successfully from a given dataset? Hint: Overfitting and underfitting in relation to size of dataset and model flexibility.

(e) What did you find frustrating and/or difficult, and/or unnecessary, and why?

(f) What would you suggest as an improvement of this assignment for next years students, and why?

(g) Did you spend more than 20 hours to complete this exercise? Why?

NOTE: Even if you are working in pairs, each person must send in a personal reflection.

(4) Mandatory Seminar: You discussing/presenting your results during the associated mandatory seminar. You do NOT have to prepare any separate slide presentation, it will be fine to show your Colab notebook if/when you are requested to present something.

10 Extra Study Material, Tips, Etc

10.1 PyTorch Help, Troubleshooting and Guidance

- Certain bugs and oddities are more common than others. Here is a link to a 20 minutes long video that gives you 10 great practical tips when you are about to start training your PyTorch models <https://www.youtube.com/watch?v=02wJ3tkc-TU>

10.2 Gradient Decent and Stochastic Gradient Decent

- "Gradient descent, how neural networks learn — Chapter 2, Deep learning" from 3Blue1Brown - https://www.youtube.com/watch?v=IHZwWFHwa-w&list=PLZHQ0b0WTQDNU6R1_67000Dx_ZCJB-3pi&index=2
- "Stochastic Gradient Descent, Clearly Explained!!!" by StatQuest - <https://www.youtube.com/watch?v=vMh0zPT0tLI>
- "25. Stochastic Gradient Descent" Suvrit Sra from MIT 18.065 Matrix Methods in Data Analysis, Signal Processing, and Machine Learning, Spring 2018 - <https://www.youtube.com/watch?v=k3AiUhWHQ28>
- "The Stochastic Gradient Descent Algorithm" by Nathan Kutz https://www.youtube.com/watch?v=_adhFSH66jc

10.3 Backpropagation to achieve auto-differentiation

- "What is backpropagation really doing? — Chapter 3, Deep learning" from 3Blue1Brown - https://www.youtube.com/watch?v=Ilg3gGewQ5U&list=PLZHQ0b0WTQDNU6R1_67000Dx_ZCJB-3pi&index=3
- "Backpropagation calculus — Chapter 4, Deep learning" from 3Blue1Brown - https://www.youtube.com/watch?v=tIeHLnjs5U8&list=PLZHQ0b0WTQDNU6R1_67000Dx_ZCJB-3pi&index=4

Good Luck and Enjoy!